FACULTY OF SCIENCE AND TECHNOLOGY
Assignment Coversheet

Student ID number: 3243463
Student Name: Ethan Fitzpatrick
Unit name: Software Technology 2
Unit number: 7170
Name of lecturer/tutor: Girija Chetty
Assignment topic: Assignment 2 - Group Project
Due date: 17/05/2026
Word Count: N/A

You must keep a photocopy or electronic copy of your assignment.

Student declaration -
I certify that the attached assignment is my own work. Material drawn from other sources has been appropriately and fully acknowledged as to author/creator, source and other bibliographic details. Such referencing may need to meet unit-specific requirements as to format and style. I give permission for my assignment to be copied, submitted and retained for the electronic checking of plagiarism.

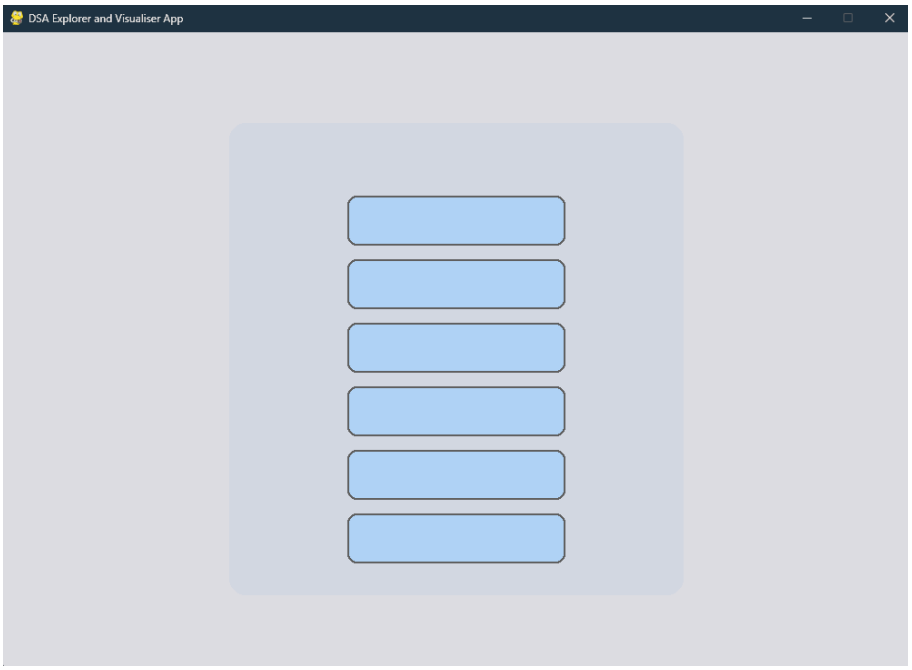
Signature of student: Ethan Fitzpatrick
Date: 17/05/2026
(Students submitting work electronically can type their name in the space for signature above, but must produce a signed copy of this coversheet on request.)
Date of submission 17/05/2026

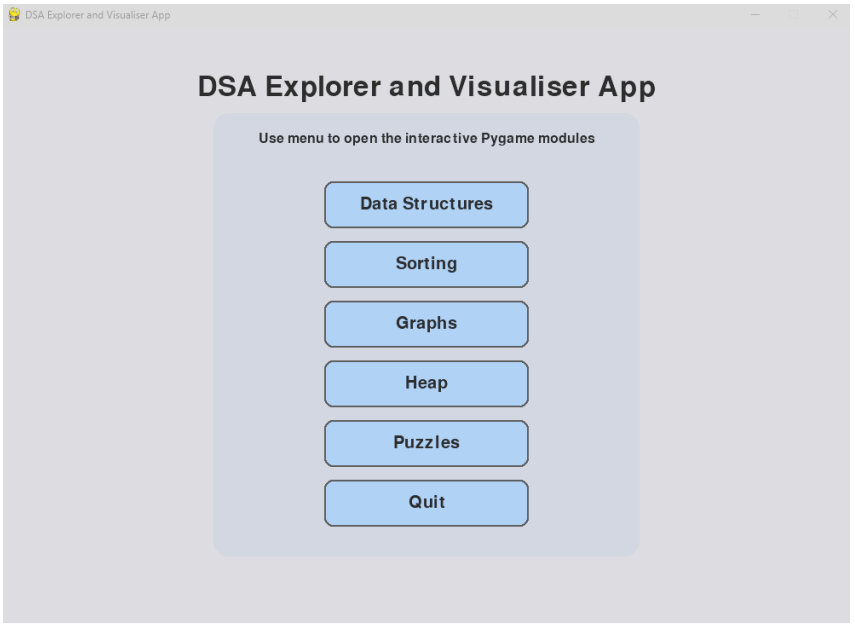
Project Overview and Main Menu -

The aim of the project was to produce a DSA Explorer and Visualiser app, that would allow users to interactively explore Data Structures and algorithms through the use of Python and Pygame. Users are able to interact with the visualiser through the use of buttons, status messages and graphical representations.

The application includes five modules: Puzzle Challenges, Heap & Event Queues, Sorting, Graphs and Data Structures. Users will access the modules via the main menu, as seen below:

1. Main menu initial state + with text added at the top giving a title, instructions below and module names within the clickable blue boxes.



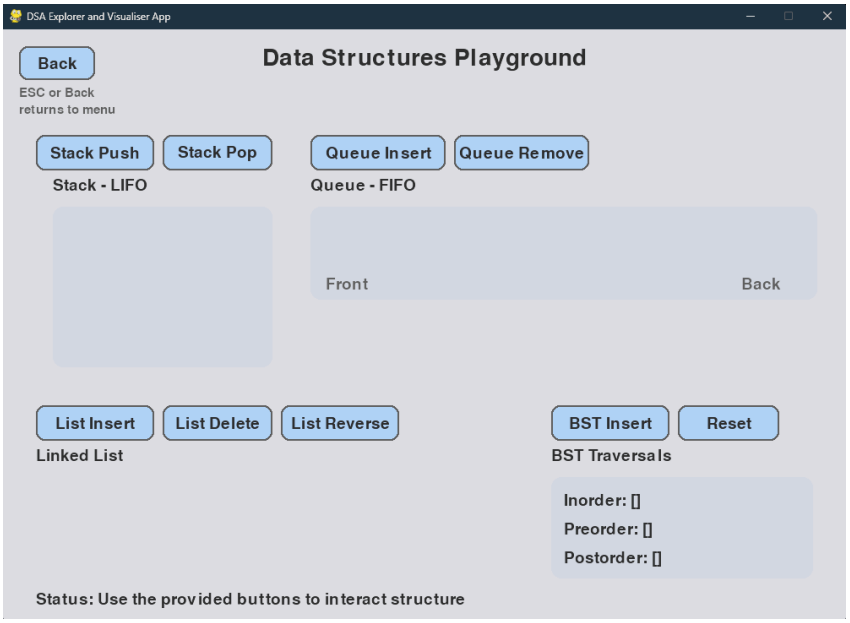


Data Structure Playground -

The data structures playground visualises four main data structure types; Stack, Queue, Linked List and Binary Search Tree (BST).

Each data structure type is visualised through the dataStructures.py file.

Initial state after moving from main menu to the Data Structures Playground:

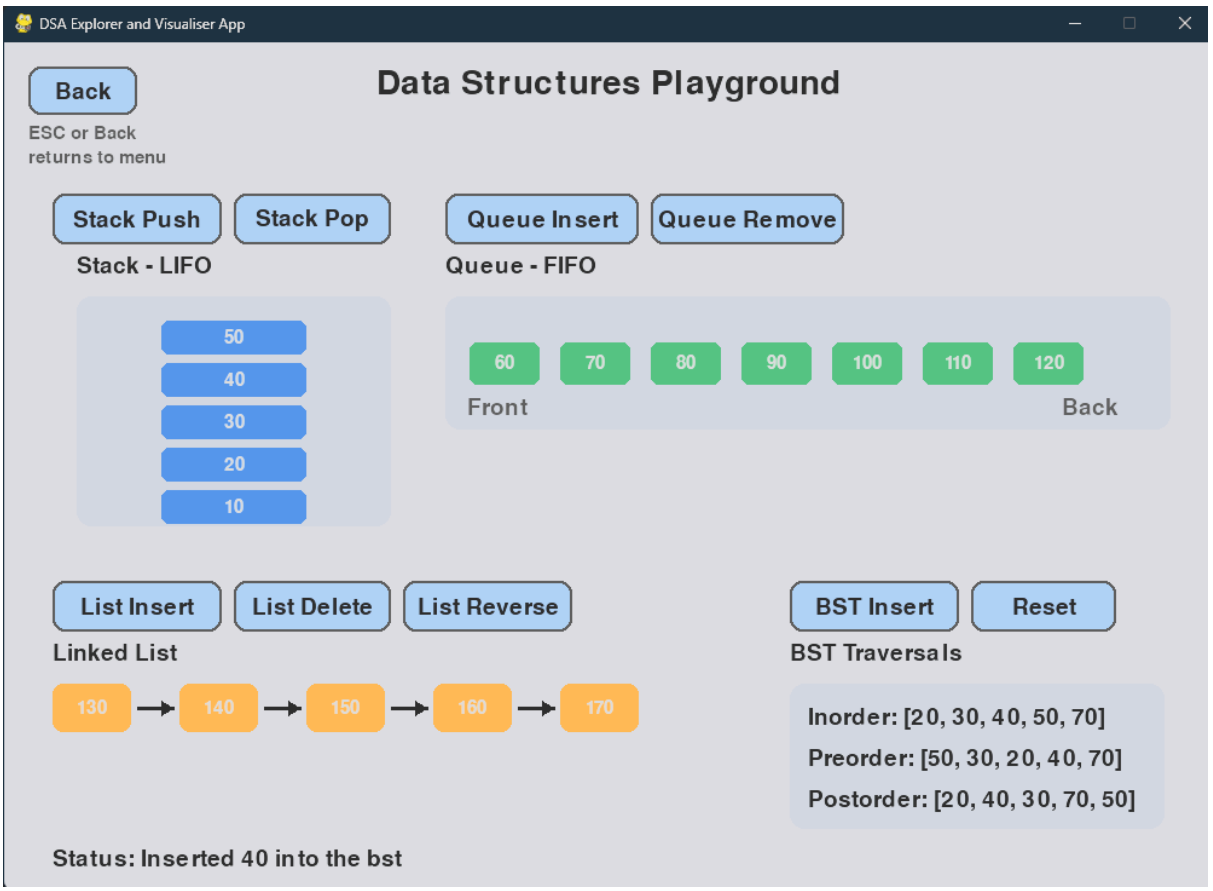


- Messaging is clearly stated within buttons, titles and status messaging.
- All buttons work correctly as intended, as seen below:

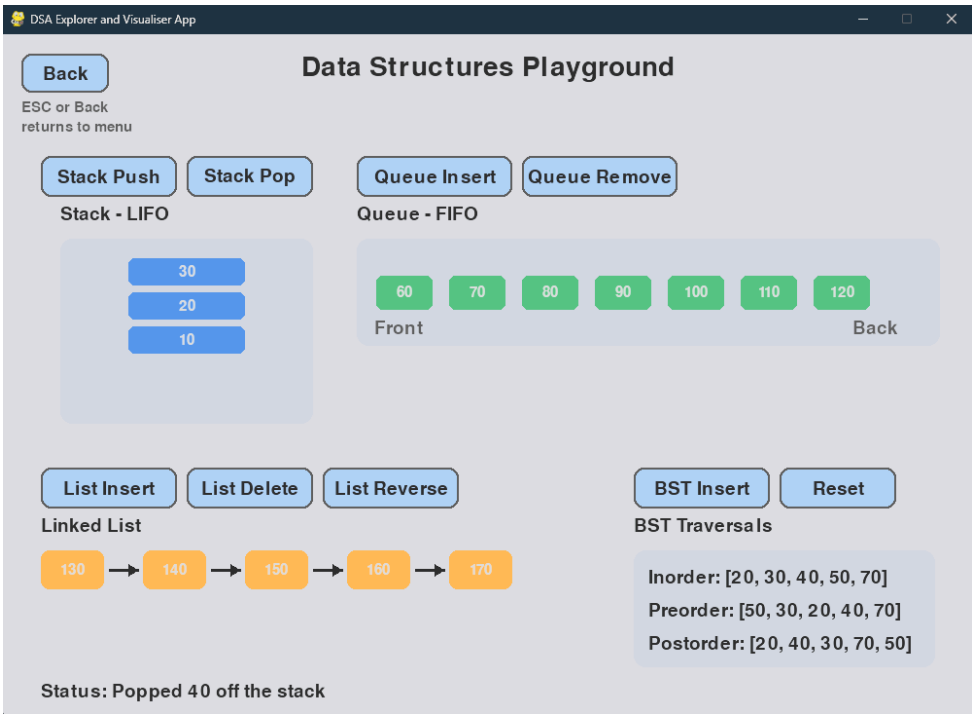
Module was developed to allow users to visualise how stack uses LIFO behaviour, when the newest value pushed is also the first that is popped. Similarly, Queue has also been visualised and shows FIFO behaviour, where the first value inserted is the first one to be removed from the list.

Linked list allows a user to insert, reverse values and delete, it also includes arrows showing the connection between nodes. BST visualises the Binary Search Tree and shows users the tree structure by inserting predetermined values. The BST data structure also shows the different traversal orders; Inorder, Preorder and Postorder.

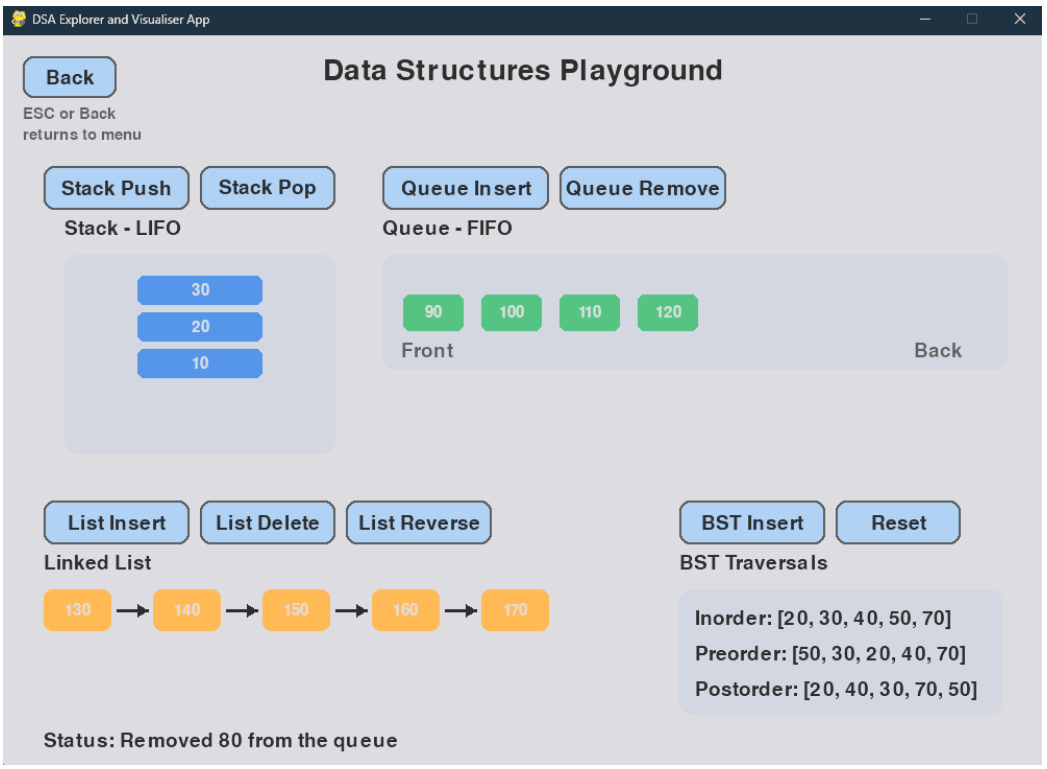
Visualisation was improved to better show users the interconnectedness within the BST, which makes it easier to understand then relying on just the traversal text.



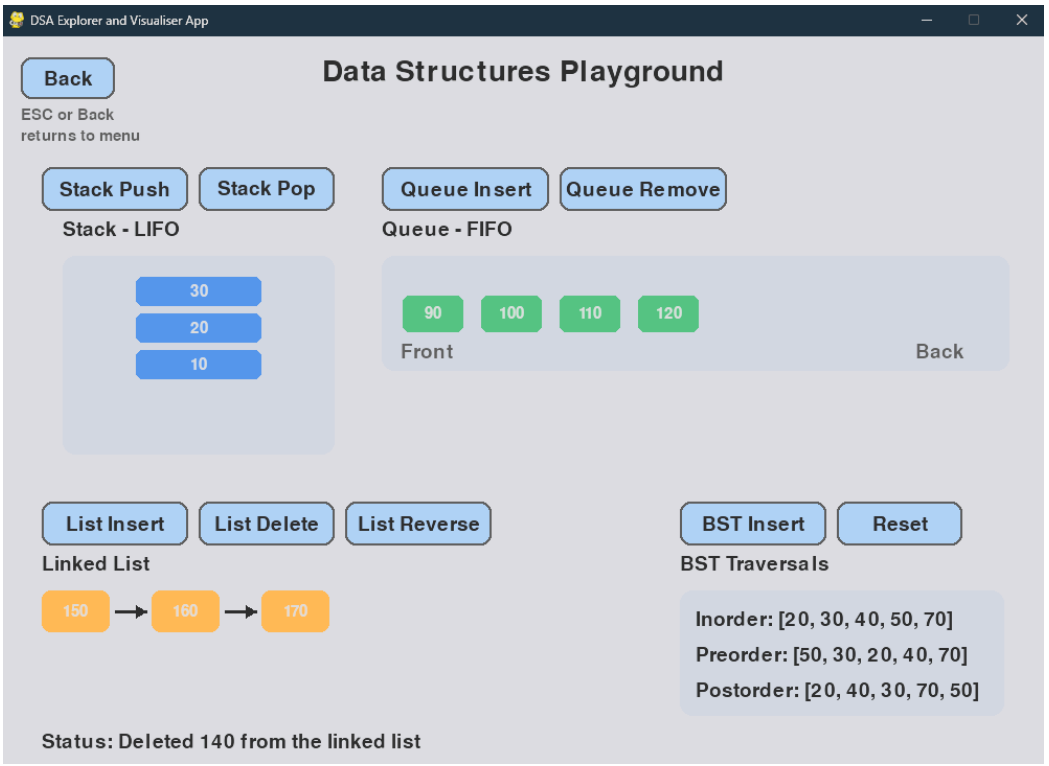
- Image shows Stack Push, Queue Insert, List Insert and BST Insert button being used. Status message updates after each button press depends on what action was taken.



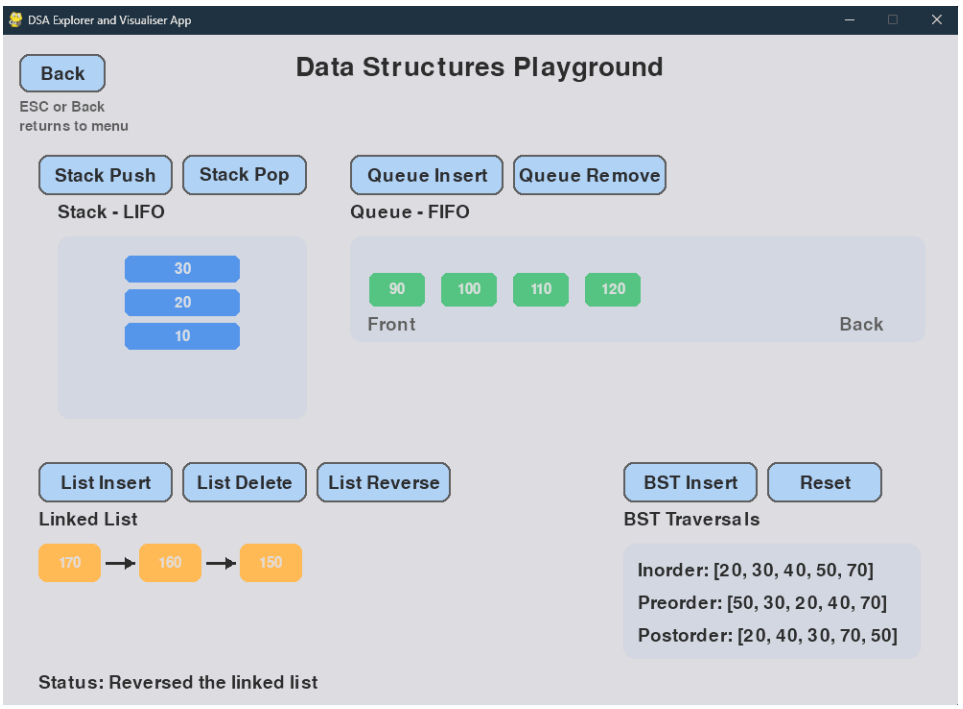
- Stack Pop used and status messaging confirms action undertaken



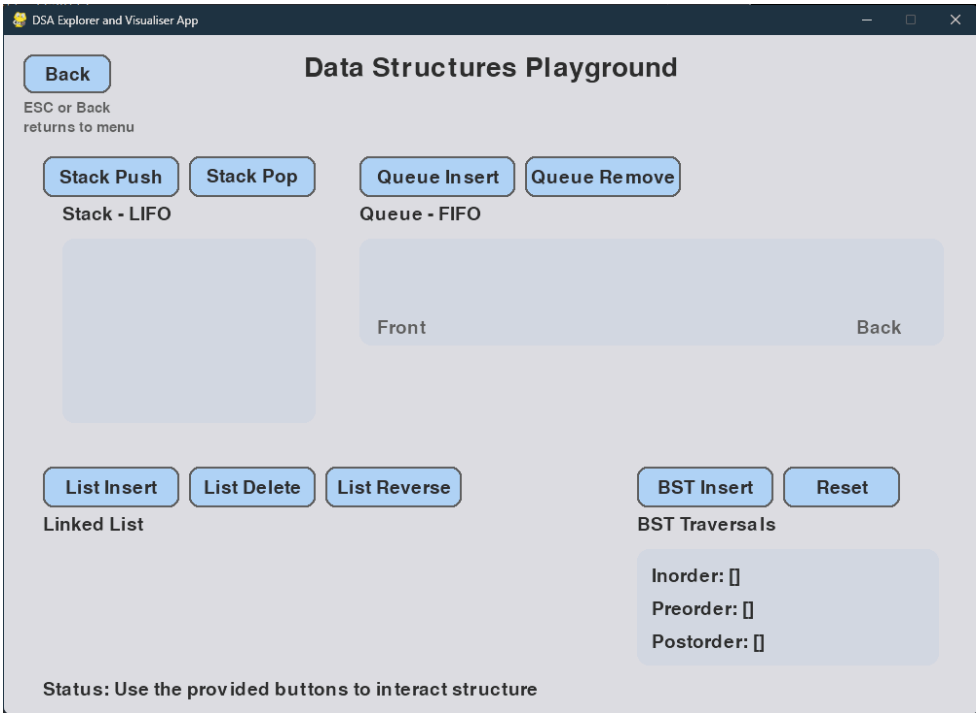
- Queue Remove button being used



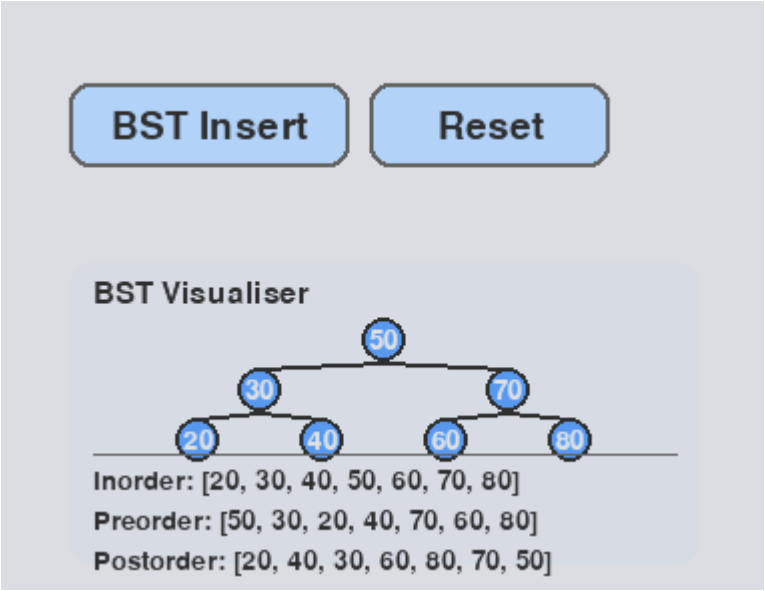
- List Delete being used



- List Reverse being used



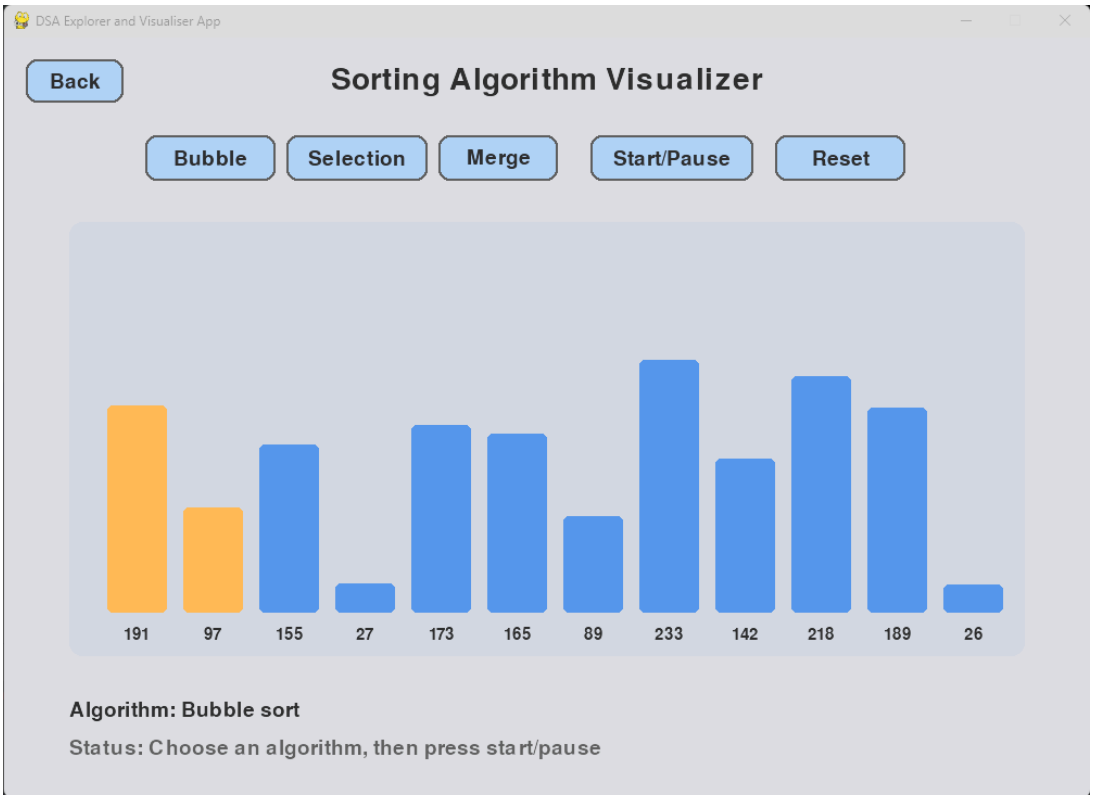
- Reset button resets the entire playground to commencement state
- Back button takes user back to main menu (user can also use the ESC button for same action)



- Updated BST Visualiser with image to better represent how Binary Search Tree works

Sorting Algorithm Visualiser -

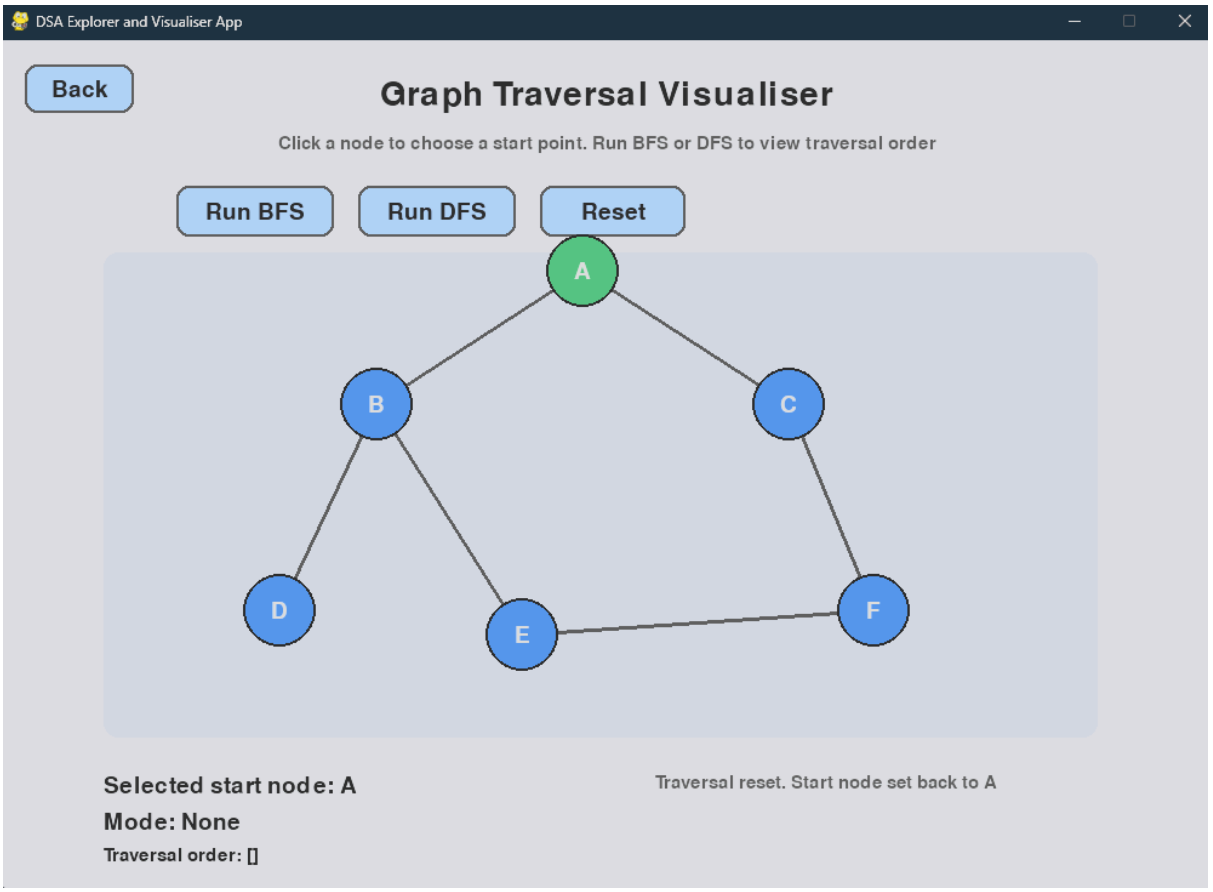
The sorting algorithm visualiser allows users to see how Bubble Sort, Selection Sort and Merge Sort work when implemented within programs. Within the visualiser, users will be able to select which sort they want to see. The program will randomly generate values to use before each sort is run.



- User can select which sorting algorithm they'd like to use at the top
- The algorithm that is currently selected will appear next to "Algorithm" at the bottom of the screen.
- The user uses the start/pause button to commence sorting using the selected algorithm.
- Reset button resets the visualiser so that a new algorithm can be selected and sorting commenced.

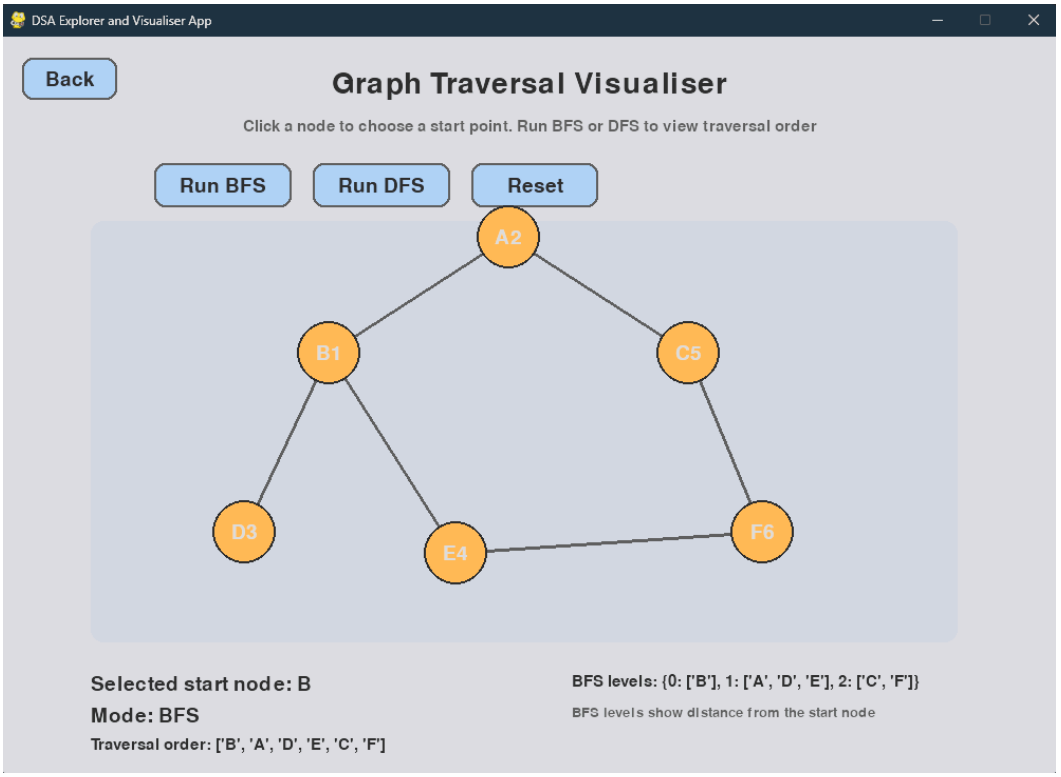
Graph Traversal Visualiser -

The graph traversal visualiser allows users to understand Breadth first search and Depth first search. A graph is displayed with nodes and connecting edges. The BFS or DFS results will be displayed down the bottom of the page, BFS will display distance from starting node, while DFS displays the depth, showing how far each of the nodes is reached during the traversal branch.



- User is provided with instructions at the top, interactively: user can click on the node they wish to commence the traversal at.
- The selected start node will update with the node that has been selected.

- User can then run BFS or DFS via the buttons provided at the top, in which case it will provide a breakdown of the traversal down the bottom:



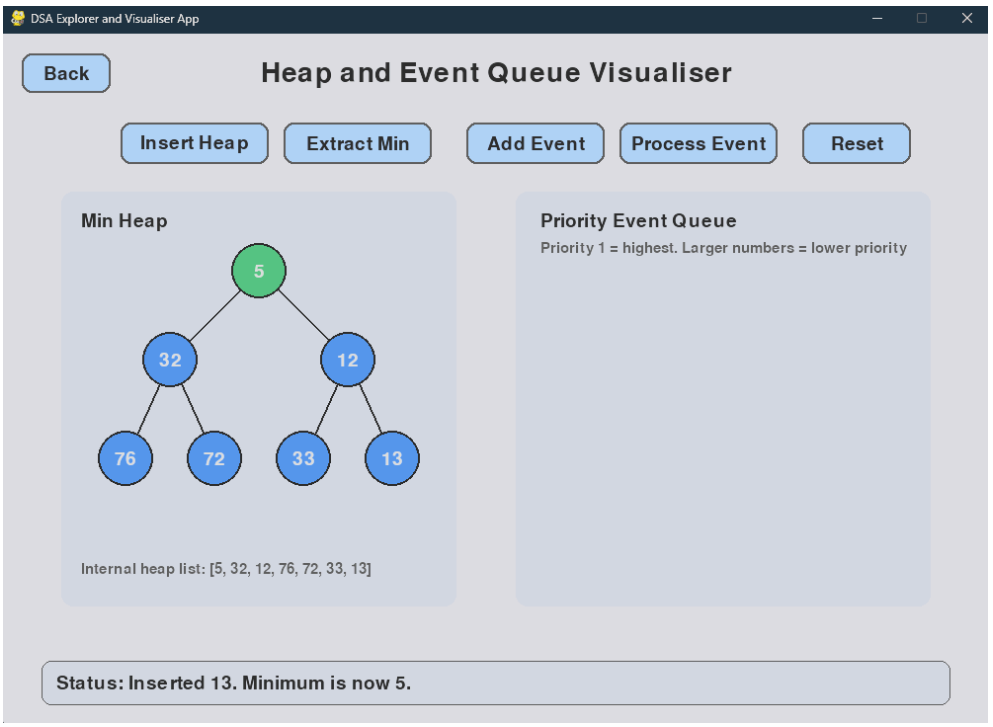
- Users can use the reset button to reset the visualiser program back to the initial state, allowing for new nodes to be selected and different algorithms to be run.

Heap Visualiser & Event Queue Simulator -

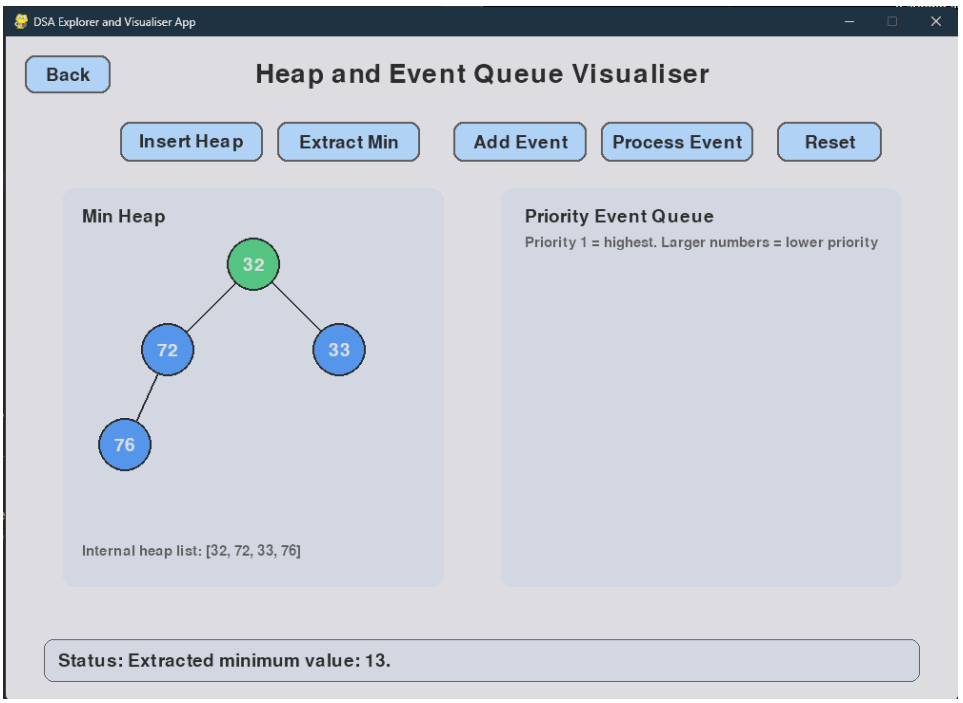
The heap visualiser demonstrates to users how Min Heap works, where the smallest value is always removed first. Users will insert random values and extract the minimum value. The heap has been visualised as a tree for better understanding.

The event queue simulator demonstrates priority event processing. Each event will be created with a priority value and time value. P1 is treated as the highest value, with each higher value having less priority the higher the number. The process event button allows users to remove the highest priority event, demonstrating the priority based event process.

- Users will find text information around the program providing instruction on how to use the visualiser.
- Users can use the back button and ESC on the keyboard as usual to exit the program. Reset button also reset the program back to the initial state.

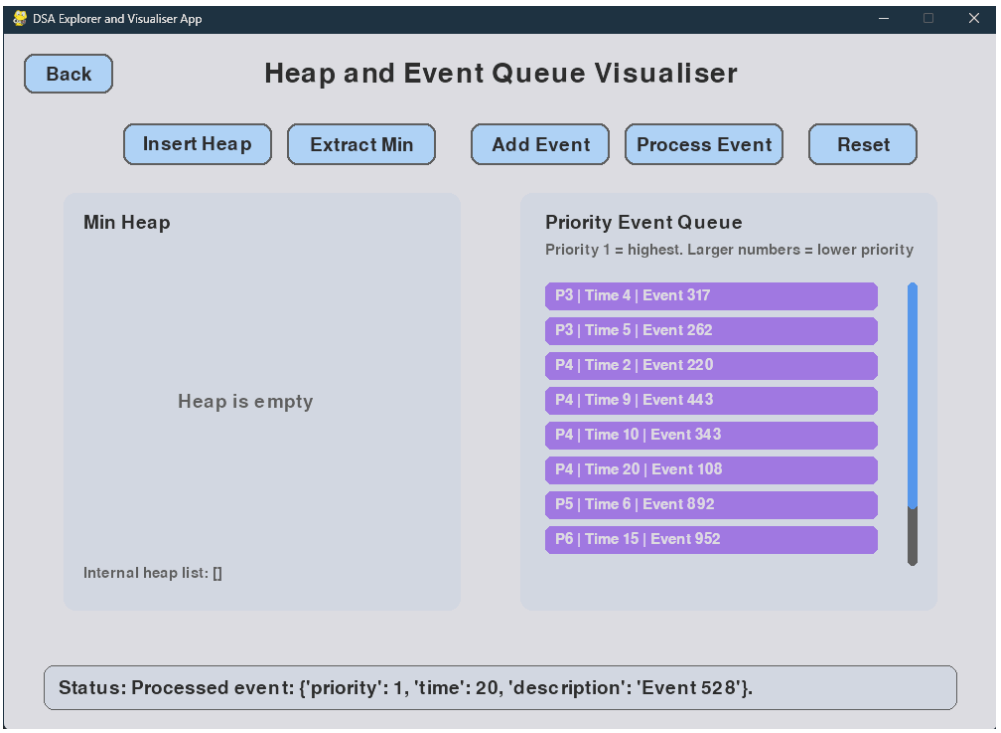


- User will use the insert heap button to insert nodes into the program, status will provide user with update on what action has occurred with each button press.



- The extract min button will extract the minimum value from the Min Heap.

- Add Event button will add events into the Priority Event Queue - provides priority number and time next to each event.
- Status message will update with the relevant information after each button press.

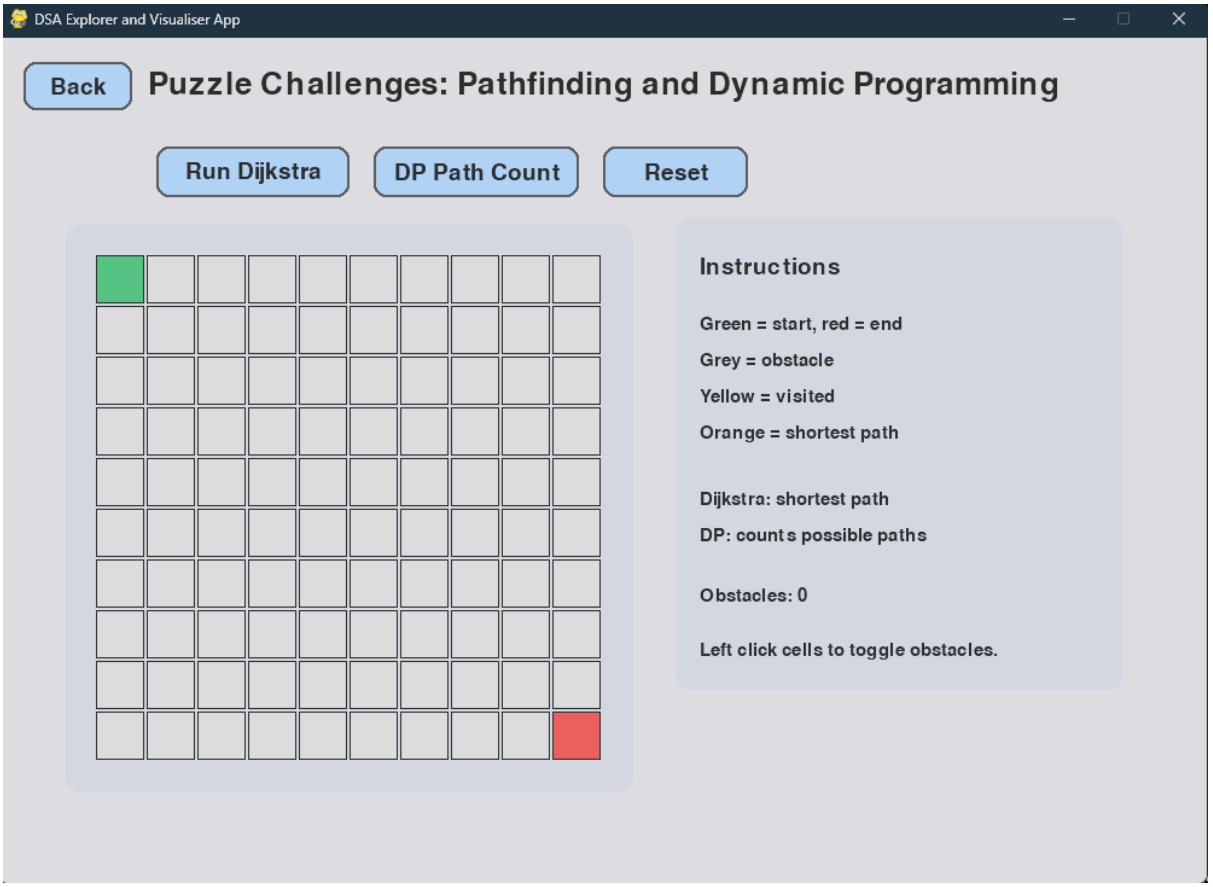


- The Process Event button will remove the event with highest priority.

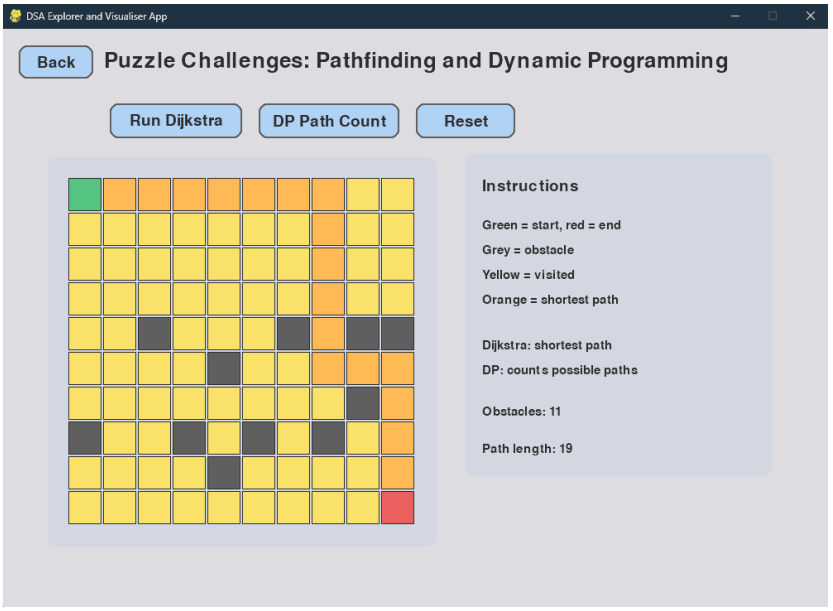
Puzzle Challenges -

Users will find the puzzle module that includes a grid space that is interactable, along with the Dijkstra pathfinding button and Dynamic Programming path counting. When the user clicks on grid spaces, they turn grey and become obstacles in the grid space. The Dijkstra algorithm will find the shortest path within the grid space, taking into account any obstacles the user lays out in the grid space.

Similarly, the Dynamic Programming path count button will calculate the number of available paths from start to end, taking into account any obstacles the user has laid out.

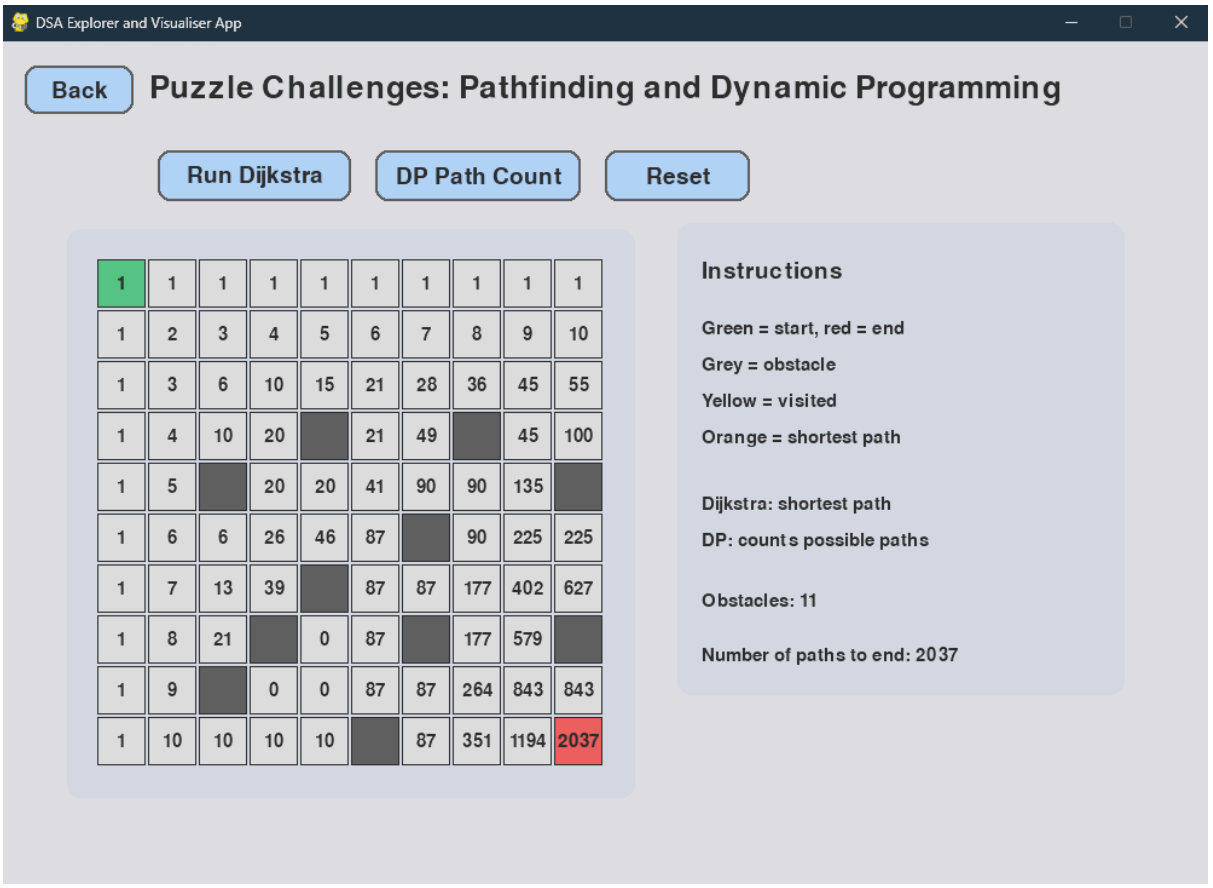


- Puzzle section will show as above when in the initial state.
- Users can interact with the grid by clicking on the individual squares. When a square is clicked, it will turn a dark grey, symbolising an obstacle.
- User can click on an obstacle within the grid to remove the obstacle.
- User can run the Dijkstra algorithm to find the shortest path available from the Start (green square) to the End (red square).



- The amount of obstacles will be recorded on the right side, along with the shortest amount of grid spaces it takes to get from the start to the end.

- Users can use the DP (Dynamic Programming) path count button to count the number of available paths within the grid space (taking into account obstacles as well).



- The number of paths to the end is shown at the end (within the red square), as well as on the right side in plain text.
- The reset button will reset the application back to its initial state.

Automated testing results -

Manual testing was completed via actively opening and running each module, checking buttons, visuals and status messages worked as intended. Screenshots provided above ere from this menial testing.

Created a separate folder to hold the automated test files. The tests were completed using Python's unittest framework, with each module having a separate test file created. Tests were used to check expected outputs automatically using assertions. A benchmarking test was also created to record timings for specific algorithms; Bubble sort, Selection sort, Merge sort, Dijkstra pathfinding and Dynamic Programming path counting.
testDataStructure -

```
PS C:\Users\Ethan\Documents\GitHub\7170_Assessment_2_DSA_Explorer_App> python -m unittest tests.testDataStructure -v
test_duplicate_values_are_ignored (tests.testDataStructure.TestBST.test_duplicate_values_are_ignored) ... ok
test_empty_tree_traversals (tests.testDataStructure.TestBST.test_empty_tree_traversals) ... ok
test_inorder_is_sorted (tests.testDataStructure.TestBST.test_inorder_is_sorted) ... ok
test_inorder_traversal (tests.testDataStructure.TestBST.test_inorder_traversal) ... ok
test_postorder_traversal (tests.testDataStructure.TestBST.test_postorder_traversal) ... ok
test_preorder_traversal (tests.testDataStructure.TestBST.test_preorder_traversal) ... ok
test_single_node_tree (tests.testDataStructure.TestBST.test_single_node_tree) ... ok
test_delete_existing_node (tests.testDataStructure.TestLinkedList.test_delete_existing_node) ... ok
test_delete_from_empty_list_returns_false (tests.testDataStructure.TestLinkedList.test_delete_from_empty_list_returns_false) ... ok
test_delete_head_node (tests.testDataStructure.TestLinkedList.test_delete_head_node) ... ok
test_delete_missing_node_returns_false (tests.testDataStructure.TestLinkedList.test_delete_missing_node_returns_false) ... ok
test_insert_at_head (tests.testDataStructure.TestLinkedList.test_insert_at_head) ... ok
test_insert_at_position (tests.testDataStructure.TestLinkedList.test_insert_at_position) ... ok
test_insert_beyond_end_appends (tests.testDataStructure.TestLinkedList.test_insert_beyond_end_appends) ... ok
test_reverse_empty_list (tests.testDataStructure.TestLinkedList.test_reverse_empty_list) ... ok
test_reverse_multiple_nodes (tests.testDataStructure.TestLinkedList.test_reverse_multiple_nodes) ... ok
test_reverse_multiple_nodes (tests.testDataStructure.TestLinkedList.test_reverse_multiple_nodes) ... ok
test_reverse_single_node (tests.testDataStructure.TestLinkedList.test_reverse_single_node) ... ok
test_traverse_empty_list (tests.testDataStructure.TestLinkedList.test_traverse_empty_list) ... ok
test_insert_remove_fifo_order (tests.testDataStructure.TestQueue.test_insert_remove_fifo_order) ... ok
test_is_empty_after_insert (tests.testDataStructure.TestQueue.test_is_empty_after_insert) ... ok
test_is_empty_on_new_queue (tests.testDataStructure.TestQueue.test_is_empty_on_new_queue) ... ok
test_items_returns_copy (tests.testDataStructure.TestQueue.test_items_returns_copy) ... ok
test_remove_empty_queue_returns_none (tests.testDataStructure.TestQueue.test_remove_empty_queue_returns_none) ... ok
test_remove_returns_front_element (tests.testDataStructure.TestQueue.test_remove_returns_front_element) ... ok
test_size_tracks_correctly (tests.testDataStructure.TestQueue.test_size_tracks_correctly) ... ok
test_is_empty_after_push (tests.testDataStructure.TestStack.test_is_empty_after_push) ... ok
test_is_empty_on_new_stack (tests.testDataStructure.TestStack.test_is_empty_on_new_stack) ... ok
test_items_returns_copy (tests.testDataStructure.TestStack.test_items_returns_copy) ... ok
test_items_returns_values_in_order (tests.testDataStructure.TestStack.test_items_returns_values_in_order) ... ok
test_lifo_order (tests.testDataStructure.TestStack.test_lifo_order) ... ok
test_peek_does_not_remove (tests.testDataStructure.TestStack.test_peek_does_not_remove) ... ok
test_peek_empty_stack_returns_none (tests.testDataStructure.TestStack.test_peek_empty_stack_returns_none) ... ok
test_pop_empty_stack_returns_none (tests.testDataStructure.TestStack.test_pop_empty_stack_returns_none) ... ok
test_push_pop_sequence_correct_order (tests.testDataStructure.TestStack.test_push_pop_sequence_correct_order) ... ok
test_push_pop_sequence_size (tests.testDataStructure.TestStack.test_push_pop_sequence_size) ... ok

----------------------------------------------------------------------
Ran 35 tests in 0.002s

OK
```

- Tests are run using prepopulated test files, and is run in the terminal with the following command: python -m unittest tests.testDataStructure -v
- testDataStructure can be interchanged for each test script, as seen below

testAlgorithms -

```
PS C:\Users\Ethan\Documents\GitHub\7170_Assessment_2_DSA_Explorer_App> python -m unittest tests.testAlgorithms -v
test_already_sorted (tests.testAlgorithms.TestBubbleSort.test_already_sorted) ... ok
test_correctness_spec_array (tests.testAlgorithms.TestBubbleSort.test_correctness_spec_array) ... ok
test_empty_list (tests.testAlgorithms.TestBubbleSort.test_empty_list) ... ok
test_final_step_has_none_indices (tests.testAlgorithms.TestBubbleSort.test_final_step_has_none_indices) ... ok
test_reverse_sorted (tests.testAlgorithms.TestBubbleSort.test_reverse_sorted) ... ok
test_single_element (tests.testAlgorithms.TestBubbleSort.test_single_element) ... ok
test_step_count_reasonable (tests.testAlgorithms.TestBubbleSort.test_step_count_reasonable) ... ok
test_swap_flag_is_boolean (tests.testAlgorithms.TestBubbleSort.test_swap_flag_is_boolean) ... ok
test_fifo_when_priority_and_time_same (tests.testAlgorithms.TestEventQueue.test_fifo_when_priority_and_time_same) ... ok
test_is_empty (tests.testAlgorithms.TestEventQueue.test_is_empty) ... ok
test_priority_beats_time (tests.testAlgorithms.TestEventQueue.test_priority_beats_time) ... ok
test_priority_one_processed_first (tests.testAlgorithms.TestEventQueue.test_priority_one_processed_first) ... ok
test_remove_empty_returns_none (tests.testAlgorithms.TestEventQueue.test_remove_empty_returns_none) ... ok
test_remove_returns_correct_fields (tests.testAlgorithms.TestEventQueue.test_remove_returns_correct_fields) ... ok
test_time_breaks_tie_when_priority_same (tests.testAlgorithms.TestEventQueue.test_time_breaks_tie_when_priority_same) ... ok
test_traverse_returns_priority_order (tests.testAlgorithms.TestEventQueue.test_traverse_returns_priority_order) ... ok
test_bfs_from_A (tests.testAlgorithms.TestGraph.test_bfs_from_A) ... ok
test_bfs_from_F (tests.testAlgorithms.TestGraph.test_bfs_from_F) ... ok
test_bfs_levels_from_A (tests.testAlgorithms.TestGraph.test_bfs_levels_from_A) ... ok
test_bfs_levels_from_F (tests.testAlgorithms.TestGraph.test_bfs_levels_from_F) ... ok
test_bfs_unknown_vertex_returns_empty (tests.testAlgorithms.TestGraph.test_bfs_unknown_vertex_returns_empty) ... ok
test_dfs_depths_from_A (tests.testAlgorithms.TestGraph.test_dfs_depths_from_A) ... ok
test_dfs_from_A (tests.testAlgorithms.TestGraph.test_dfs_from_A) ... ok
test_dfs_from_C (tests.testAlgorithms.TestGraph.test_dfs_from_C) ... ok
test_dfs_unknown_vertex_returns_empty (tests.testAlgorithms.TestGraph.test_dfs_unknown_vertex_returns_empty) ... ok
test_edges_added (tests.testAlgorithms.TestGraph.test_edges_added) ... ok
test_no_self_loops_allowed (tests.testAlgorithms.TestGraph.test_no_self_loops_allowed) ... ok
test_vertices_added (tests.testAlgorithms.TestGraph.test_vertices_added) ... ok
test_insert_and_peek_minimum (tests.testAlgorithms.TestHeap.test_insert_and_peek_minimum) ... ok
test_is_empty (tests.testAlgorithms.TestHeap.test_is_empty) ... ok
test_peek_empty_returns_none (tests.testAlgorithms.TestHeap.test_peek_empty_returns_none) ... ok
test_remove_empty_returns_none (tests.testAlgorithms.TestHeap.test_remove_empty_returns_none) ... ok
test_remove_returns_minimum (tests.testAlgorithms.TestHeap.test_remove_returns_minimum) ... ok
test_repeated_remove_returns_sorted_values (tests.testAlgorithms.TestHeap.test_repeated_remove_returns_sorted_values) ... ok
test_correctness (tests.testAlgorithms.TestMergeSort.test_correctness) ... ok
test_duplicates (tests.testAlgorithms.TestMergeSort.test_duplicates) ... ok
test_empty_list (tests.testAlgorithms.TestMergeSort.test_empty_list) ... ok
test_large_random_list (tests.testAlgorithms.TestMergeSort.test_large_random_list) ... ok
test_mergesortactions_final_step_sorted (tests.testAlgorithms.TestMergeSort.test_mergesortactions_final_step_sorted) ... ok
test_single_element (tests.testAlgorithms.TestMergeSort.test_single_element) ... ok
test_already_sorted (tests.testAlgorithms.TestSelectionSort.test_already_sorted) ... ok
test_correctness (tests.testAlgorithms.TestSelectionSort.test_correctness) ... ok
test_final_step_has_none_indices (tests.testAlgorithms.TestSelectionSort.test_final_step_has_none_indices) ... ok
test_no_swap_steps_for_sorted_input (tests.testAlgorithms.TestSelectionSort.test_no_swap_steps_for_sorted_input) ... ok
test_reverse_sorted (tests.testAlgorithms.TestSelectionSort.test_reverse_sorted) ... ok
test_swap_steps_exist_for_unsorted_input (tests.testAlgorithms.TestSelectionSort.test_swap_steps_exist_for_unsorted_input) ... ok

----------------------------------------------------------------------
Ran 46 tests in 0.003s

OK
```

puzzleTest -

```
PS C:\Users\Ethan\Documents\GitHub\7170_Assessment_2_DSA_Explorer_App> python -m unittest tests.puzzleTest -v
test_no_path_when_blocked (tests.puzzleTest.TestDijkstra.test_no_path_when_blocked) ... ok
test_path_avoids_obstacles (tests.puzzleTest.TestDijkstra.test_path_avoids_obstacles) ... ok
test_path_includes_start_and_end_only_once (tests.puzzleTest.TestDijkstra.test_path_includes_start_and_end_only_once) ... ok
test_path_is_contiguous (tests.puzzleTest.TestDijkstra.test_path_is_contiguous) ... ok
test_path_length_no_obstacles (tests.puzzleTest.TestDijkstra.test_path_length_no_obstacles) ... ok
test_simple_path_no_obstacles (tests.puzzleTest.TestDijkstra.test_simple_path_no_obstacles) ... ok
test_start_equals_end (tests.puzzleTest.TestDijkstra.test_start_equals_end) ... ok
test_visit_order_starts_at_start (tests.puzzleTest.TestDijkstra.test_visit_order_starts_at_start) ... ok
test_2x2_no_obstacles (tests.puzzleTest.TestGridPaths.test_2x2_no_obstacles) ... ok
test_3x3_no_obstacles (tests.puzzleTest.TestGridPaths.test_3x3_no_obstacles) ... ok
test_left_column_all_ones_when_no_obstacles (tests.puzzleTest.TestGridPaths.test_left_column_all_ones_when_no_obstacles) ... ok
test_obstacle_at_end_zero_paths (tests.puzzleTest.TestGridPaths.test_obstacle_at_end_zero_paths) ... ok
test_obstacle_at_start_zero_paths (tests.puzzleTest.TestGridPaths.test_obstacle_at_start_zero_paths) ... ok
test_obstacle_cell_is_zero (tests.puzzleTest.TestGridPaths.test_obstacle_cell_is_zero) ... ok
test_obstacle_reduces_count (tests.puzzleTest.TestGridPaths.test_obstacle_reduces_count) ... ok
test_single_cell_grid (tests.puzzleTest.TestGridPaths.test_single_cell_grid) ... ok
test_single_cell_grid_blocked (tests.puzzleTest.TestGridPaths.test_single_cell_grid_blocked) ... ok
test_top_row_all_ones_when_no_obstacles (tests.puzzleTest.TestGridPaths.test_top_row_all_ones_when_no_obstacles) ... ok

----------------------------------------------------------------------
Ran 18 tests in 0.001s

OK
```

User can also run a benchmarking script with prepopulated data that tests BST, Dijkstra, Dynamic Programming paths, Merge Sort, Selection Sort and also compares the different sort algorithms against one another -

```
PS C:\Users\Ethan\Documents\GitHub\7170_Assessment_2_DSA_Explorer_App> python -m unittest tests.Benchmarking -v
test_benchmark_bubble_sort_steps (tests.Benchmarking.TestBenchmarks.test_benchmark_bubble_sort_steps) ...
--- Bubble sort visualisation-step benchmarks ---
n=  25 | average=0.05 ms
n=  50 | average=0.37 ms
n= 100 | average=3.16 ms
ok
test_benchmark_dijkstra (tests.Benchmarking.TestBenchmarks.test_benchmark_dijkstra) ...
--- Dijkstra grid pathfinding benchmarks ---
10x10 grid | average=0.09 ms
20x20 grid | average=0.38 ms
30x30 grid | average=0.83 ms
ok
test_benchmark_grid_paths (tests.Benchmarking.TestBenchmarks.test_benchmark_grid_paths) ...
--- Dynamic programming grid path benchmarks ---
10x10 grid | average=0.01 ms
20x20 grid | average=0.04 ms
50x50 grid | average=0.27 ms
ok
test_benchmark_merge_sort (tests.Benchmarking.TestBenchmarks.test_benchmark_merge_sort) ...
--- Merge sort benchmarks ---
n= 100 | average=0.07 ms
n= 500 | average=0.39 ms
n=1000 | average=0.84 ms
ok
test_benchmark_selection_sort_steps (tests.Benchmarking.TestBenchmarks.test_benchmark_selection_sort_steps) ...
--- Selection sort visualisation-step benchmarks ---
n=  25 | average=0.03 ms
n=  50 | average=0.18 ms
n= 100 | average=1.91 ms
ok
test_sort_comparison_summary (tests.Benchmarking.TestBenchmarks.test_sort_comparison_summary) ...
--- Sort comparison summary ---
Bubble sort visual steps    : 0.0094s
Selection sort visual steps : 0.0058s
Merge sort                  : 0.0002s
ok

----------------------------------------------------------------------
Ran 6 tests in 0.045s

OK
```

Conclusion -

In conclusion, the DSA Explore and Visualiser app performed well given the milestones and requirements that were given in the assignment instructions. The app, using Pygame, provides an interactive experience for the user to better understand data structures, sorting algorithms, graph traversal, heap & event queue simulation and have some fun puzzle challenges.